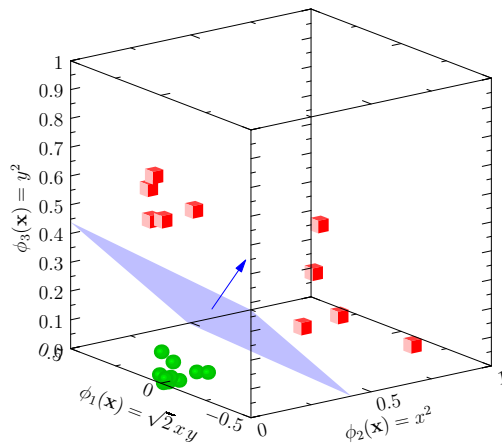




The Kernel Trick, SVMs, Regression

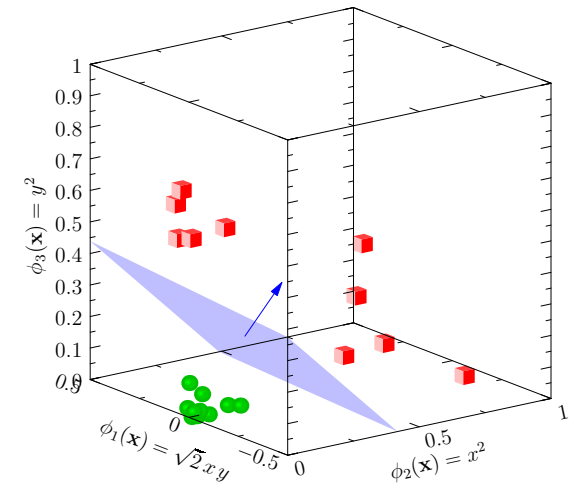
SVM Kernels

- SVM Kernels are functions of two variables that can be factorised

$$K(x, y) = \langle \phi(x), \phi(y) \rangle = \sum_i \phi_i(x) \phi_i(y)$$

- where $\phi(x) = (\phi_1(x), \phi_2(x), \dots)^T$ and $\phi_i(x)$ are real valued functions of x
- $K(x, y)$ will be positive semi-definite (because it is an inner-product)
- Furthermore, any positive semi-definite function will factorise
- This factorisation is not always obvious (we return to this later)

- The Kernel Trick
- Positive Semi-Definite Kernels
- Kernel Properties
- Beyond Classification



Dual Form

- Recall that the dual problem for an SVM is

$$\max_{\alpha} \sum_{k=1}^m \alpha_k - \frac{1}{2} \sum_{k,l=1}^m \alpha_k \alpha_l y_k y_l \langle \phi(x_k), \phi(x_l) \rangle$$

- subject to $\sum_{k=1}^m y_k \alpha_k = 0$ and $0 \leq \alpha_k (\leq C)$
- But since $K(x_k, x_l) = \langle \phi(x_k), \phi(x_l) \rangle$ the dual problem becomes

$$\max_{\alpha} \sum_{k=1}^m \alpha_k - \frac{1}{2} \sum_{k,l=1}^m \alpha_k \alpha_l y_k y_l K(x_k, x_l)$$

- This is the **kernel trick**—we never have to compute $\phi(x)$

Classifying New Data

- Having trained the SVM we now have to use it
- Given a new input x we decide on the class

$$y = \text{sgn}(\langle w, \phi(x) \rangle - b) \quad \text{but} \quad w = \sum_{k=1}^m \alpha_k y_k \phi(x_k)$$

- In the dual representation this becomes

$$\text{sgn}\left(\sum_{k=1}^m \alpha_k y_k K(x_k, x) - b\right)$$

where we only need to sum over the non-zero α_k (i.e. the support vectors SVs)

Recap on Eigen Systems

- Recall for a symmetric ($n \times n$) matrix M an eigenvector, v

$$Mv = \lambda v$$

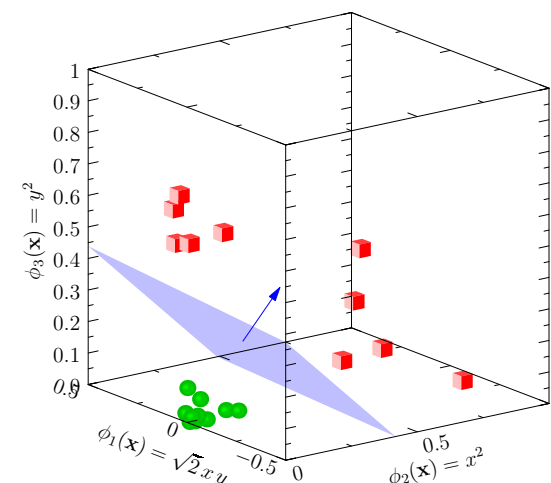
- There are n independent eigenvectors $v^{(i)}$ with real eigenvalues $\lambda^{(i)}$
- The eigenvectors are orthogonal so that $v^{(i)\top} v^{(j)} = 0$ if $i \neq j$
- Forming a matrix of eigenvectors $V = (v^{(1)}, v^{(2)}, \dots, v^{(n)})$ the matrix satisfies

$$V^T V = V V^T = I$$

- Such matrices are said to be orthogonal

Outline

1. The Kernel Trick
2. **Positive Semi-Definite Kernels**
3. Kernel Properties
4. Beyond Classification



Eigen Decomposition

- From the eigenvalue equation $Mv^{(k)} = \lambda^{(k)}v^{(k)}$

$$MV = V\Lambda \quad \text{where} \quad \Lambda = \begin{pmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{pmatrix}$$

- Multiplying on the right by V^T we get

$$M = V\Lambda V^T = \sum_{k=1}^n \lambda^{(k)} v^{(k)} v^{(k)\top}$$

Or

$$M_{ij} = \sum_{k=1}^n \lambda^{(k)} v_i^{(k)} v_j^{(k)} = \sum_{k=1}^n u_i^{(k)} u_j^{(k)} = \langle u_i, u_j \rangle$$

$$u_i^{(k)} = \sqrt{\lambda^{(k)}} v_i^{(k)}$$

Eigenfunctions

- By analogy for a symmetric function of two variables we can define an *eigenfunction*

$$\int K(x, y) \psi(y) dy = \lambda \psi(x)$$

- In general there will be a denumerable set of eigenfunctions $\psi^{(k)}(x)$ where

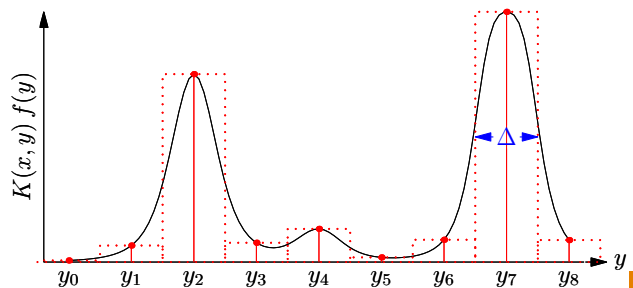
$$K(x, y) = \sum_k \lambda^{(k)} \psi^{(k)}(x) \psi^{(k)}(y)$$

- This is known as Mercer's theorem

Linear Operators

- Recall a linear function $\mathcal{T}[f(x)]$ can be represented by a kernel

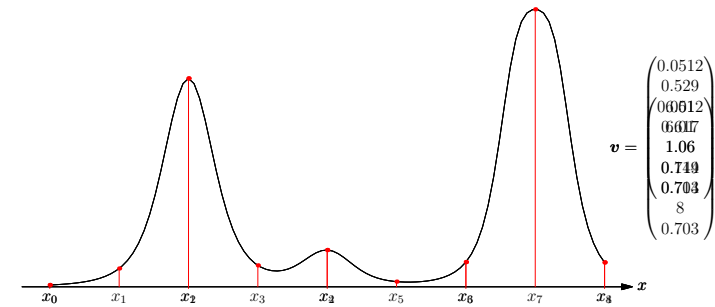
$$\mathcal{T}[f(x)] = \int_{y \in \mathcal{I}} K(x, y) f(y) dy \approx \Delta \sum_{j=1}^n K(x, y_j) f(y_j)$$



This is just a matrix equation with $M_{ij} = \Delta K(x_i, y_j)$

Limit Process

- Consider sampling a function at a set of points



- In the limit where the number of sample points goes to infinity the vector more closely approximates a function
- The k^{th} component of the vector corresponds to $f(x_k)$

SVM Kernels

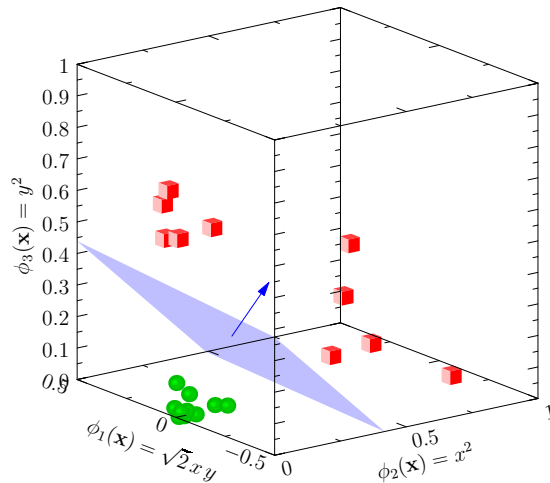
- If we define $\phi^{(k)}(x) = \sqrt{\lambda^{(k)}} \psi^{(k)}(x)$ then

$$K(x, y) = \sum_k \lambda^{(k)} \psi^{(k)}(x) \psi^{(k)}(y) = \sum_k \phi^{(k)}(x) \phi^{(k)}(y)$$

- This is the definition of a SVM kernel we started with
- Note that for $\phi^{(k)}(x)$ to be real $\lambda^{(k)} \geq 0$ for all k
- If $\lambda^{(k)} < 0$ then $\langle \phi(x), \phi(x) \rangle = \|\phi(x)\|^2$ might be negative and “distance” between points in the extended feature space can be negative
- If we use a kernel that isn't positive semi-definite then the Hessian of the dual objective function will not be negative semi-definite and there will be a maximum where α diverges

Outline

1. The Kernel Trick
2. Positive Semi-Definite Kernels
3. **Kernel Properties**
4. Beyond Classification



Positive Semi-Definite Kernels

- Kernels (or matrices) that have eigenvalues $\lambda^{(k)} \geq 0$ are called positive semi-definite
- (If the eigenvalues are strictly positive $\lambda^{(k)} > 0$ the kernels or matrices are called positive definite)
- Positive semi-definite kernels can always be decomposed into a sum of real functions

$$K(x, y) = \langle \phi(x), \phi(y) \rangle$$

Properties of Positive Semi-Definiteness

- Since

$$K(x, y) = \langle \phi(x), \phi(y) \rangle$$

- An immediate consequence is that for any function $f(x)$

$$\begin{aligned} \int f(x) K(x, y) f(y) dx dy &= \int f(x) \langle \phi(x), \phi(y) \rangle f(y) dx dy \\ &= \left\langle \int f(x) \phi(x) dx, \int f(y) \phi(y) dy \right\rangle \\ &= \left\| \int f(x) \phi(x) dx \right\|^2 \geq 0 \end{aligned}$$

Positive Semi-Definiteness

- The following statements are equivalent
 - ★ $K(x, y)$ is positive semi-definite (written $K(x, y) \succeq 0$)
 - ★ The eigenvalues of $K(x, y)$ are non-negative
 - ★ The kernel can be written

$$K(x, y) = \langle \phi(x), \phi(y) \rangle$$

where the $\phi^{(k)}(x)$'s are real functions

- ★ For any real function $f(x)$

$$\int f(x) K(x, y) f(y) dx dy \geq 0$$

Adding Kernels

- We can construct SVM kernels from other kernels
- If $K_1(\mathbf{x}, \mathbf{y})$ and $K_2(\mathbf{x}, \mathbf{y})$ are valid kernels then so is $K_3(\mathbf{x}, \mathbf{y}) = K_1(\mathbf{x}, \mathbf{y}) + K_2(\mathbf{x}, \mathbf{y})$

$$\begin{aligned} Q &= \int f(\mathbf{x}) K_3(\mathbf{x}, \mathbf{y}) f(\mathbf{y}) d\mathbf{x} d\mathbf{y} \\ &= \int f(\mathbf{x}) (K_1(\mathbf{x}, \mathbf{y}) + K_2(\mathbf{x}, \mathbf{y})) f(\mathbf{y}) d\mathbf{x} d\mathbf{y} \\ &= \int f(\mathbf{x}) K_1(\mathbf{x}, \mathbf{y}) f(\mathbf{y}) d\mathbf{x} d\mathbf{y} + \int f(\mathbf{x}) K_2(\mathbf{x}, \mathbf{y}) f(\mathbf{y}) d\mathbf{x} d\mathbf{y} \geq 0 \end{aligned}$$

- If $K(\mathbf{x}, \mathbf{y})$ is a valid kernel so is $cK(\mathbf{x}, \mathbf{y})$ for $c > 0$

Exponentiating Kernels

- If $K(\mathbf{x}, \mathbf{y})$ is a valid kernel so is $K^n(\mathbf{x}, \mathbf{y})$ (by induction)
 - ★ Assume $K(\mathbf{x}, \mathbf{y}) \succeq 0$ this satisfies base case
 - ★ If $K^{n-1}(\mathbf{x}, \mathbf{y}) \succeq 0$ then

$$K^n(\mathbf{x}, \mathbf{y}) = K^{n-1}(\mathbf{x}, \mathbf{y}) K(\mathbf{x}, \mathbf{y}) \succeq 0$$

- and $\exp(K(\mathbf{x}, \mathbf{y}))$ is also a valid kernel since

$$e^{K(\mathbf{x}, \mathbf{y})} = \sum_{i=1}^{\infty} \frac{1}{i!} K^i(\mathbf{x}, \mathbf{y}) = 1 + K(\mathbf{x}, \mathbf{y}) + \frac{1}{2} K^2(\mathbf{x}, \mathbf{y}) + \dots$$

but each term in the sum is a kernel

Product of Kernels

- If $K_1(\mathbf{x}, \mathbf{y})$ and $K_2(\mathbf{x}, \mathbf{y})$ are valid kernels then so is $K_3(\mathbf{x}, \mathbf{y}) = K_1(\mathbf{x}, \mathbf{y}) K_2(\mathbf{x}, \mathbf{y})$
- Writing

$$K_1(\mathbf{x}, \mathbf{y}) = \sum_i \phi_i^{(1)}(\mathbf{x}) \phi_i^{(1)}(\mathbf{y}), \quad K_2(\mathbf{x}, \mathbf{y}) = \sum_j \phi_j^{(2)}(\mathbf{x}) \phi_j^{(2)}(\mathbf{y})$$

then

$$\begin{aligned} K_3(\mathbf{x}, \mathbf{y}) &= \sum_{i,j} \phi_i^{(1)}(\mathbf{x}) \phi_i^{(1)}(\mathbf{y}) \phi_j^{(2)}(\mathbf{x}) \phi_j^{(2)}(\mathbf{y}) \\ &= \sum_{i,j} \phi_{ij}^{(3)}(\mathbf{x}) \phi_{ij}^{(3)}(\mathbf{y}) = \langle \phi^{(3)}(\mathbf{x}), \phi^{(3)}(\mathbf{y}) \rangle \end{aligned}$$

where $\phi_{ij}^{(3)}(\mathbf{x}) = \phi_i^{(1)}(\mathbf{x}) \phi_j^{(2)}(\mathbf{x})$

RBF Kernel

- Now $\mathbf{x}^T \mathbf{y} = \langle \mathbf{x}, \mathbf{y} \rangle$ is a valid kernel because it is an inner product of functions $\phi(\mathbf{x}) = \mathbf{x}$
- For $\gamma > 0$ we have $2\gamma \mathbf{x}^T \mathbf{y} \succeq 0$
- Thus $\exp(2\gamma \mathbf{x}^T \mathbf{y}) \succeq 0$
- If $K(\mathbf{x}, \mathbf{y}) \succeq 0$ then $g(\mathbf{x}) K(\mathbf{x}, \mathbf{y}) g(\mathbf{y}) \succeq 0$

$$\int f(\mathbf{x}) g(\mathbf{x}) K(\mathbf{x}, \mathbf{y}) g(\mathbf{y}) f(\mathbf{y}) d\mathbf{x} d\mathbf{y} = \int h(\mathbf{x}) K(\mathbf{x}, \mathbf{y}) h(\mathbf{y}) d\mathbf{x} d\mathbf{y} \geq 0$$

where $f(\mathbf{x}) g(\mathbf{x}) = h(\mathbf{x})$

$$e^{-\gamma \mathbf{x}^T \mathbf{x}} e^{2\gamma \mathbf{x}^T \mathbf{y}} e^{-\gamma \mathbf{y}^T \mathbf{y}} = e^{-\gamma \|\mathbf{x} - \mathbf{y}\|^2} \succeq 0$$

Other Kernels

- The success of SVMs has meant that researchers try to increase the area of application
- The condition that a SVM kernel must be positive semi-definite is quite restrictive
- There has been a cottage industry of researchers finding smart kernels for solving complicated problems
- The key to finding new kernels is to use the properties of kernels to build more complicated kernels from simpler ones

String Kernels

- One area where SVMs were very important is in document classification
- This requires comparing strings
- There are a large number of kernels developed to do this

Spectrum Kernel

- A simple way to compare documents is to collect a histogram of all occurrences of substrings of length p
- This is known as a p -spectrum
- A p -spectrum kernel counts the number of common substrings

$s = \text{statistics}$ $\mathcal{S}_3(s) = \{\text{sta,tat,ati,tis,ist,sti,tic,ics}\}$
 $t = \text{computation}$ $\mathcal{S}_3(t) = \{\text{com,omp,mpu,put,uta,tat,ati,tio,ion}\}$

- $K(s,t) = 2$ (“tat” and “ati”)

All Subsequences Kernel

- A more sophisticated kernel is to count all of the common subsequences that occur in two documents
- Naively this would take an exponential amount of time to compute
- Using clever dynamic-programming techniques this can be done relatively efficiently
- This can even be extended to include sub-sequence matches with possible gaps between words

Other Kernel Applications

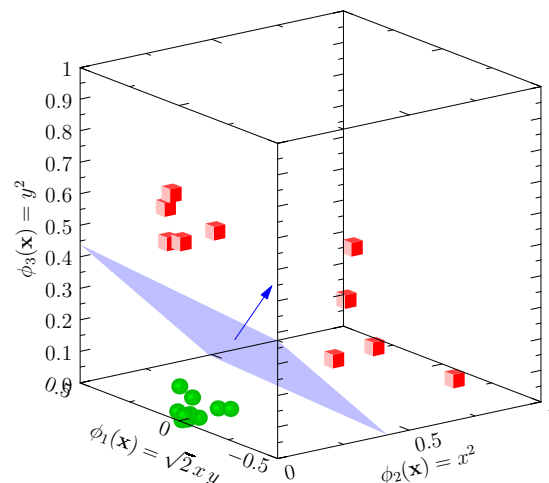
- String kernels for comparing subsequences are used in bioinformatics
- Kernels have been developed for comparing trees (e.g. for computer program evaluation, XML, etc.)
- Kernels have also been developed for comparing graphs (e.g. for comparing chemicals based on their molecular graph)

Fisher Kernels

- In an attempt to build kernels that capture more domain knowledge, kernels are constructed from other learning machines
- An example of this are “Fisher kernels” whose features come from an Hidden Markov Model (HMM) trained on the data
- These tend to have better discriminative power than the underlying model (HMM), and has a better feature set than a SVM using a generic kernel

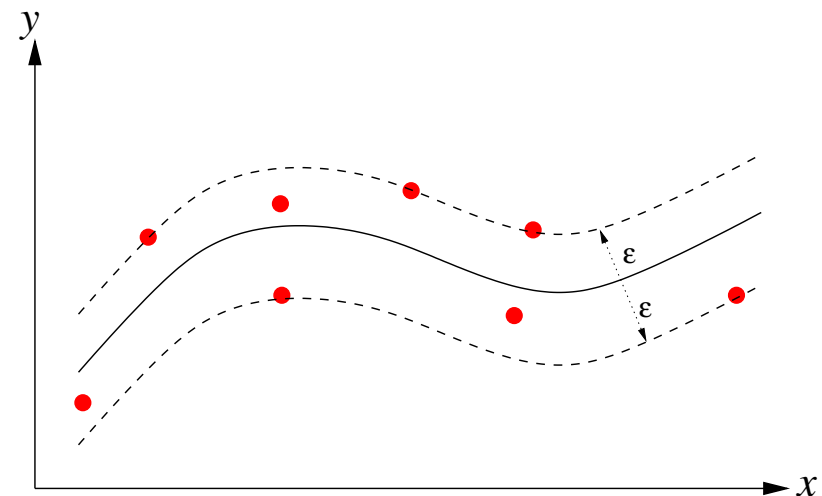
Outline

1. The Kernel Trick
2. Positive Semi-Definite Kernels
3. Kernel Properties
4. **Beyond Classification**



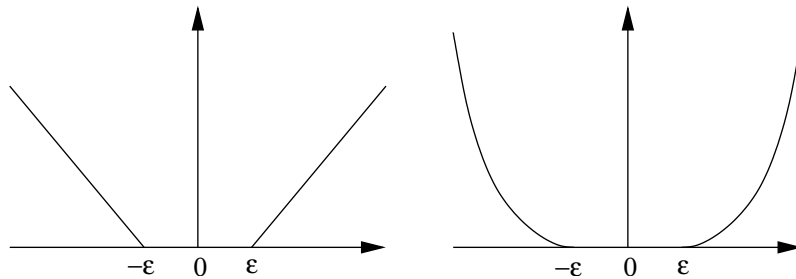
Regression with Margins

- SVMs can be modified to perform regression



Error Functions

- Can introduce slack variables with different errors



- This can be transformed to a quadratic programming problem

Kernel Methods

- Kernel methods where we project into an extended feature space are used with other linear algorithms
 - ★ Kernel Fisher discriminant analysis (KFDA)
 - ★ Kernel principle component analysis (KPCA)
 - ★ Kernel canonical correlation analysis (KCCA)
 - ★ Gaussian Processes
- These are also extremely powerful machine learning algorithms

Ridge Regression Using Kernels

- We can also solve regression problems without using margins
- To solve a regression problem once again the problem is set up as a quadratic programming problem

$$\min_{\mathbf{w}} \lambda \|\mathbf{w}\|^2 + \sum_{i=1}^m (y_i - \mathbf{w}^T \phi(\mathbf{x}_i))^2$$

- the $\|\mathbf{w}\|^2$ is a regularisation term
- By assuming $\mathbf{w} = \sum_i \alpha_i \phi(\mathbf{x}_i)$ we obtain a quadratic equation for the α_i 's which we can solve

Summary

- SVMs require a positive definite kernel function
- These can be built from simpler functions
- There was a cottage industry of people creating new kernels for different applications
- SVMs are just one example of a host of machines that
 - ★ use the kernel trick
 - ★ often use linear constraints
 - ★ tend to be convex optimisation problems